# AlgoBench

## Algorithm Benchmarking & Profiling Suite

SSL Project

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

| | |
|---|---|
| CS25BT005 | Sorting and Searching |
| CS25BT022 | Graph Algorithms |
| CS25BT031 | Dynamic Programming and Matrix Algorithms |
| CS25BT041 | Benchmarking and Profiling |
| CS25BT051 | Dashboard and Integration |

November 2026

# Contents

**Abstract** **1**

**1 Introduction** **2**
1.1 Problem Statement . . . 2
1.2 Proposed Solution . . . 2
1.3 Objectives . . . 2
1.4 Scope . . . 2

**2 System Design** **3**
2.1 High-Level Architecture . . . 3
2.2 Project Modules . . . 3
2.3 Algorithm Catalogue . . . 3
2.4 Dataset Design . . . 4

**3 Benchmarking and Profiling Methodology** **5**
3.1 Runtime Measurement . . . 5
3.2 Memory Measurement . . . 5
3.3 Function Profiling . . . 5
3.4 CSV Schema . . . 6
3.5 GNUplot . . . 6

**4 Implementation** **7**
4.1 Dashboard Workflow . . . 7
4.2 Benchmark Engine Interface . . . 7
4.3 Project Automation . . . 7

**5 Testing and Validation** **8**
5.1 Automated Test Coverage . . . 8
5.2 Manual Validation Checklist . . . 8

**6 Results and Discussion** **9**
6.1 Experimental Environment and Protocol . . . 9
6.2 Sorting . . . 9
6.3 Searching . . . 10
6.4 Graph Algorithms . . . 11
6.5 Dynamic Programming . . . 11
6.6 Matrix Multiplication . . . 12
6.7 Profiler Observations . . . 13
6.8 Overall Findings . . . 13

**7 Limitations and Future Work** **15**
7.1 Current Limitations . . . . . 15
7.2 Future Work . . . . . 15

**8 Conclusion** **16**

**Team Contributions** **17**

**References** **18**

**AI Tool Usage Declaration** **19**

# Abstract

Algorithm implementations are often studied through their asymptotic time complexities, but observed performance also depends on input characteristics, implementation choices, memory allocation, and runtime overhead. AlgoBench is a Python-based algorithm benchmarking and profiling suite with a Streamlit dashboard. It allows users to select algorithms, generate reproducible input datasets, execute repeated measurements, inspect memory usage and profiling reports, and compare performance as input size changes.

The project integrates sixteen implementations across sorting, searching, graph algorithms, dynamic programming, and matrix multiplication. A reusable benchmark engine measures function execution time using `time.perf_counter()`, estimates additional Python allocations using `tracemalloc`, and optionally profiles function-level costs using `cProfile` and `pstats`. CSV files provide persistent experiment history, while GNUplot scripts prepare runtime and memory visualizations.

The unified automated test suite reports 158 passing pytest cases and 64 subtests in the integration environment. The dashboard was also exercised on the project computer across all five domains. This report includes the CSV measurements and GNUplot runtime and memory figures produced during those local experiments. The measurements are specific to the recorded Windows and CPython environment and should not be interpreted as universal timings.

# Chapter 1

# Introduction

## 1.1 Problem Statement

Students commonly implement algorithms and compare them by theoretical complexity. However, theoretical bounds alone do not show how implementations behave for particular datasets or on a real execution environment. Two algorithms with the same asymptotic complexity may have different constant factors and memory demands. An algorithm may also behave differently on random, sorted, reverse-sorted, sparse, or dense inputs.

A repeatable way is therefore needed to generate controlled datasets, execute algorithms under consistent measurement conditions, inspect bottlenecks, and visualize the observed results.

## 1.2 Proposed Solution

AlgoBench provides a single interface for configuring algorithm experiments. It connects algorithm implementations and dataset generators to a generic benchmark engine. The dashboard presents timing statistics, memory estimates, profile reports, CSV history, and comparison charts.

## 1.3 Objectives

1. Implement and organize algorithms from five computational domains.
2. Generate deterministic datasets using configurable random seeds.
3. Measure execution time over repeated trials.
4. Measure additional Python memory allocations separately from timing.
5. Use profiling tools to identify expensive functions.
6. Compare observed performance over multiple input sizes.
7. Provide CSV storage, history browsing, and runtime/memory plotting.
8. Demonstrate modular development through Git, tests, Bash scripts, and a Makefile.

## 1.4 Scope

The current implementation supports sixteen algorithm variants in five categories. It focuses on reproducible Python experiments. Native profiling support is included as an optional gprof demonstration for C programs; it is separate from Python function profiling.

# Chapter 2

# System Design

## 2.1 High-Level Architecture

The application follows a modular pipeline:

1. **Streamlit dashboard:** accepts the user's category, algorithm, dataset, trial, seed, memory, and profiling choices.
2. **Algorithm registry:** maps dashboard labels to Python callables and prepares the correct positional arguments.
3. **Dataset generators:** create reproducible arrays, graphs, strings, item lists, and matrices.
4. **Benchmark engine:** invokes registered callables and records timing, memory, and profiling metadata.
5. **Storage and visualization:** writes CSV files, prepares GNUplot data, and displays comparison charts and history.

## 2.2 Project Modules

| Module | Responsibility |
|---|---|
| `algorithms/` | Sorting, searching, graph, dynamic programming, and matrix implementations. |
| `data_generation/` | Reproducible graph, string, and matrix datasets; array and item datasets are prepared by the registry. |
| `benchmarking/` | Algorithm registry, benchmark engine, timer, memory profiler, cProfile, and gprof integration. |
| `visualization/` | Streamlit charts and GNUplot data/plot generation. |
| `tests/` | Unit, generator, benchmark-engine, and integration tests. |
| `scripts/` | Bash helpers for running the app, tests, and GNUplot. |
| `reports/` | LaTeX report and Beamer source. |

## 2.3 Algorithm Catalogue

| Domain | Algorithms | Representative time complexity |
|---|---|---|

| | | |
|---|---|---|
| Sorting | Quick Sort, Merge Sort, Heap Sort, Insertion Sort | Quick Sort: average $O(n \log n)$, worst $O(n^2)$; Merge/Heap: $O(n \log n)$; Insertion: average/-worst $O(n^2)$. |
| Searching | Linear Search, Binary Search, Hash Table Lookup | $O(n)$; $O(\log n)$ on ascending-sorted input; average $O(1)$ lookup. |
| Graph | Dijkstra, Bellman–Ford, BFS | $O((V+E) \log V)$; $O(VE)$; $O(V+E)$. |
| Dynamic Programming | LCS memoization/tabulation; 0/1 Knapsack memoization/tabulation | LCS: $O(mn)$; Knapsack: $O(nW)$. |
| Matrix | Standard Multiplication, Strassen Multiplication | $O(n^3)$; approximately $O(n^{2.807})$ for square matrices. |

## 2.4 Dataset Design

The dataset generator uses a supplied seed to make experiments reproducible.

- Sorting supports random, sorted, reverse-sorted, and nearly-sorted arrays.
- Searching supports successful and unsuccessful queries. When Binary Search is included in a comparison, the shared collection is made ascending-sorted.
- Graph experiments use connected, undirected weighted graphs with sparse or dense topology.
- LCS receives a pair of generated strings. Knapsack receives weights, values, and a capacity derived from the item count.
- Matrix algorithms receive the same pair of compatible square matrices for a given size and seed.

BFS reports shortest unweighted edge-count distances and ignores graph edge weights. Dijkstra and Bellman–Ford use the positive edge weights generated by the dashboard dataset factory.

# Chapter 3

# Benchmarking and Profiling Methodology

## 3.1 Runtime Measurement

The benchmark engine uses Python's monotonic high-resolution performance counter:

```
start = time.perf_counter()
result = algorithm(*prepared_args)
elapsed = time.perf_counter() - start
```

An untimed warm-up call is performed before the timed trials. For each algorithm and dataset, the engine records:

- average execution time,
- median execution time,
- minimum and maximum execution time,
- standard deviation, and
- individual raw trial timings in the in-memory result.

Dataset generation and input preparation/copying occur outside the timed interval. The measurement therefore represents the algorithm call itself.

## 3.2 Memory Measurement

Memory is measured separately using `tracemalloc`. The recorded value is peak additional traced Python allocation in KiB during one algorithm call. It is not the total RAM usage of the Python process. Running memory tracing inside timed trials would distort runtime measurements, so the engine uses a separate memory execution.

## 3.3 Function Profiling

The engine optionally uses `cProfile` and `pstats` to collect function call counts, self time, cumulative time, and a textual report. Profile runs are separate from timing trials. The dashboard profiles the largest selected input size for each chosen algorithm to keep the experiment practical.

The optional `gprof_profiler.py` module demonstrates profiling of native C programs compiled with `gcc -pg`. This is distinct from `cProfile`, which profiles Python functions.

## 3.4 CSV Schema

Each saved row includes timestamp, category, algorithm, input size, input type, trial count, average/median/minimum/maximum time, standard deviation, peak memory measurement, seed, and notes. The memory unit is KiB and runtime units are seconds.

## 3.5 GNUplot

The GNUplot runner reads the CSV schema, pivots rows by category and input type, prepares tab-separated `.dat` files, and invokes runtime and memory plot scripts. Preparing data files does not require GNUplot; rendering PNG plots requires the GNUplot executable to be installed and available on the system `PATH`.

# Chapter 4

# Implementation

## 4.1 Dashboard Workflow

The Run Benchmark page validates the selection and provides practical input size ranges. Users may run one size or enable a scaling experiment over multiple sizes. A progress indicator is updated after each algorithm-size job. The resulting rows are stored in `results/` as a timestamped CSV.

The Compare Results page displays metrics and a runtime chart and provides CSV download and GNUplot actions. The Profiler page shows the latest cProfile summaries and saved reports. The History page loads experiment CSV files saved in previous sessions.

## 4.2 Benchmark Engine Interface

The engine is algorithm-agnostic. A callable is registered under a category and display name, and input data is passed as either a single argument or a tuple of positional arguments.

```
engine = create_default_engine(results_dir="results")
row = engine.run(
    "Sorting",
    "Merge Sort",
    dataset,
    input_size=1000,
    input_type="Random",
    trials=5,
    measure_memory=True,
    profile=True,
    seed=42,
)
```

This separation allows algorithm implementations to be integrated without placing algorithm-specific logic inside the benchmark engine.

## 4.3 Project Automation

The repository includes Bash scripts for running the Streamlit application, running tests, and generating GNUplot plots. A Makefile provides targets for installation, app execution, testing, plotting, and cache cleanup.

# Chapter 5

# Testing and Validation

## 5.1 Automated Test Coverage

The merged test suite includes tests for algorithm correctness, input validation, dataset reproducibility, benchmark timing, memory profiling, cProfile output, CSV persistence, GNUplot data preparation, algorithm registration, and execution through the common engine interface.

| **Validation item** | **Observed result** |
|---|---|
| Python source files syntax-checked | 35 |
| Unified pytest cases passed | 158 |
| Subtests passed | 64 |
| Direct smoke test of registered algorithms | 16 / 16 |
| GNUplot data-file preparation smoke test | Passed |

The test results above are from the integration environment used to assemble the project. The Streamlit browser interface was launched on the project computer. All five algorithm domains were run through the dashboard, CSV files were saved, and GNUplot runtime and memory PNGs were rendered for each domain.

## 5.2 Manual Validation Checklist

1. Start the application using `streamlit run app.py`.
2. Enter the dashboard through the animated logo landing page.
3. Switch Bright and Dark themes.
4. Run one small experiment from each of the five categories.
5. Run a scaling experiment with at least three input sizes.
6. Verify that each run creates a CSV in `results/`.
7. Open Compare Results, Profiler, and History.
8. Render GNUplot runtime and memory PNG plots for all five domains.

# Chapter 6

# Results and Discussion

## 6.1 Experimental Environment and Protocol

The benchmark evidence was generated on the project team's Windows computer. The profiler report paths identify CPython 3.14; the CPU model and hardware configuration were not included in the supplied evidence, so they are not reported here. Each datapoint is the average of five timed trials. The benchmark engine performs an untimed warm-up and measures the algorithm call separately from dataset construction and input preparation. Memory is measured in a separate `tracemalloc` execution and is reported as peak additional traced Python allocation in KiB.

The scaling datasets included random integer arrays for sorting, successful queries over ascending-sorted collections for searching, sparse connected weighted graphs for graph algorithms, random strings and random item sets for dynamic programming, and random square matrices for matrix multiplication. All comparisons use the input-size conventions shown in the dashboard. In particular, the searching experiment excludes hash-table construction from the timed Hash Table Lookup operation, and BFS measures unweighted edge-count distance rather than weighted shortest paths.

## 6.2 Sorting

Four sorting algorithms were measured on random arrays of 100, 500, 1,000, 2,000, and 5,000 elements. Table 6.1 reports average runtime in milliseconds.

Table 6.1: Sorting runtime by input size (milliseconds; five trials per point).

| $n$ | Quick Sort | Merge Sort | Heap Sort | Insertion Sort |
|---|---|---|---|---|
| 100 | 0.249 | 0.397 | 0.416 | 0.274 |
| 500 | 0.941 | 1.858 | 1.404 | 5.469 |
| 1,000 | 1.890 | 3.480 | 3.890 | 34.258 |
| 2,000 | 5.563 | 6.146 | 8.891 | 115.178 |
| 5,000 | 12.839 | 18.572 | 26.990 | 540.030 |

At 5,000 elements, Insertion Sort took approximately 0.540 seconds, compared with 0.0128 seconds for Quick Sort. This is consistent with the expected quadratic growth of insertion sort on random input, while the other three implementations grew more slowly over the tested range. Quick Sort was the fastest of the four in the largest-size run. These are observed implementation results on one machine, not guarantees for every input distribution.

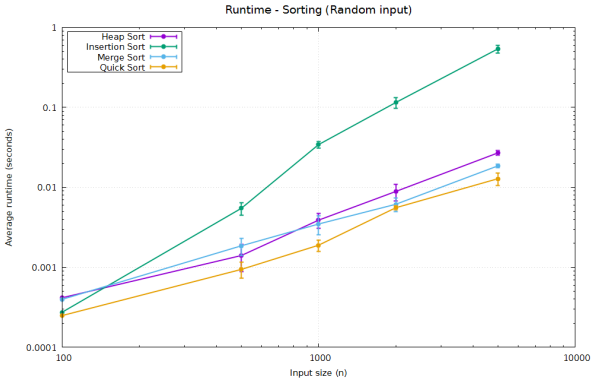



Figure 6.1: Sorting runtime scaling.

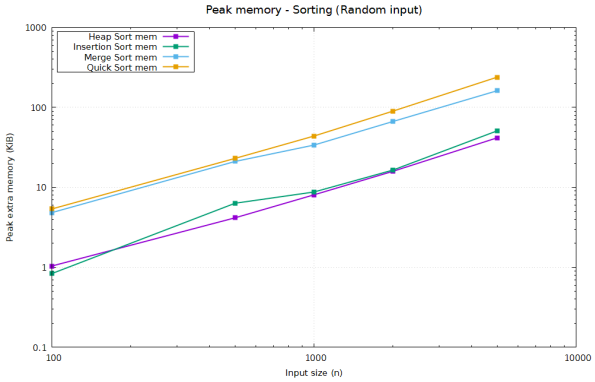



Figure 6.2: Sorting peak additional memory.

## 6.3 Searching

Linear Search, Binary Search, and Hash Table Lookup were compared for successful queries on ascending-sorted collections of 1,000, 5,000, 10,000, and 50,000 elements. Table 6.2 shows average runtime in milliseconds.

Table 6.2: Searching runtime by collection size (milliseconds; five trials).

| $n$ | Linear Search | Binary Search | Hash Table Lookup |
|---|---|---|---|
| 1,000 | 0.0901 | 0.0131 | 0.00538 |
| 5,000 | 0.3560 | 0.0117 | 0.00432 |
| 10,000 | 0.7300 | 0.0119 | 0.00526 |
| 50,000 | 3.2961 | 0.0148 | 0.00338 |

Linear Search increased from about 0.090 ms at 1,000 elements to 3.296 ms at 50,000 elements. Binary Search remained around 0.012–0.015 ms, and the measured hash lookup remained around 0.003–0.005 ms. The hash lookup result covers lookup only; table construction is outside the timed operation. The reported 0.0 KiB memory values for Binary Search and Hash Table Lookup mean that no additional traced allocation was observed in that isolated call at the recorded precision, not that these algorithms require zero total memory.

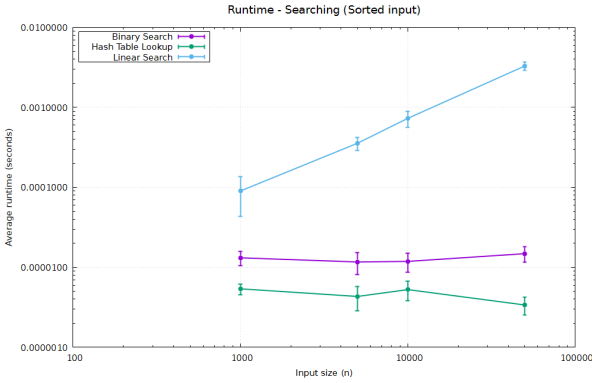



Figure 6.3: Searching runtime scaling.

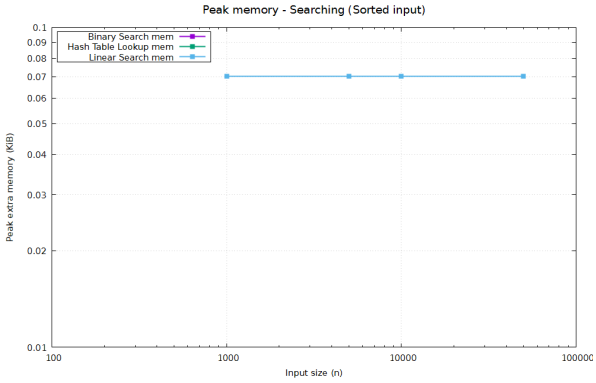



Figure 6.4: Searching peak additional memory.

## 6.4 Graph Algorithms

Dijkstra, Bellman–Ford, and BFS were tested on sparse connected weighted graphs with 20, 50, 100, and 200 vertices. Each point represents five timed trials. Table 6.3 reports milliseconds.

Table 6.3: Sparse graph runtime by vertex count (milliseconds; five trials).

| Vertices | Dijkstra | Bellman–Ford | BFS |
|---|---|---|---|
| 20 | 0.110 | 0.141 | 0.134 |
| 50 | 0.190 | 0.678 | 0.136 |
| 100 | 0.324 | 0.839 | 0.231 |
| 200 | 0.781 | 1.051 | 0.535 |

All three runtimes increased over the tested vertex counts. Bellman–Ford was the slowest at 50, 100, and 200 vertices in this scaling run. Dijkstra was faster than Bellman–Ford on the larger tested graphs. BFS was also fast, but its result is not a like-for-like weighted shortest-path comparison: BFS returns unweighted edge-count distances and ignores edge weights. The 50-vertex point has relatively high trial variability for Bellman–Ford, as reflected by its recorded standard deviation, so the difference at that size should be interpreted cautiously.

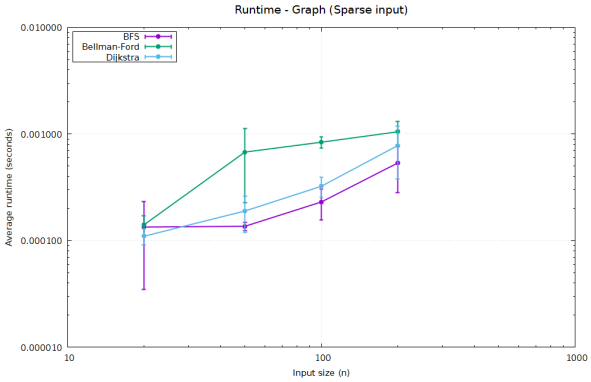



Figure 6.5: Sparse graph runtime scaling.

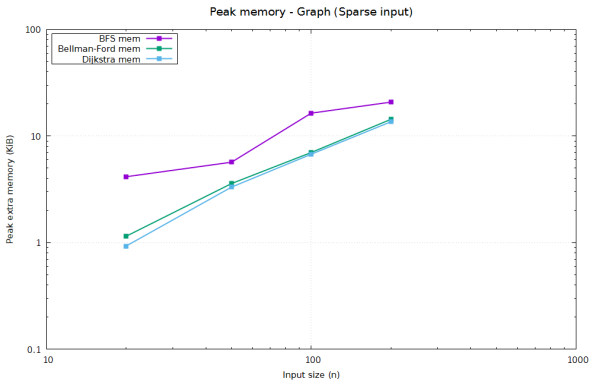



Figure 6.6: Sparse graph peak additional memory.

## 6.5 Dynamic Programming

The dynamic-programming experiments were split into two workload families: LCS on pairs of random strings and 0/1 Knapsack on random item sets. Both families were measured at input sizes 10, 20, 40, 60, 80, and 100. Each algorithm-size point contains five trials. Table 6.4 reports average runtime in milliseconds.

Table 6.4: Dynamic-programming runtime (milliseconds; five trials).

| $n$ | LCS Memo. | LCS Tab. | Knapsack Memo. | Knapsack Tab. |
|---|---|---|---|---|
| 10 | 0.169 | 0.059 | 0.214 | 0.196 |
| 20 | 0.691 | 0.133 | 1.102 | 0.514 |
| 40 | 1.613 | 0.383 | 4.803 | 2.582 |
| 60 | 2.680 | 0.743 | 10.892 | 5.310 |
| 80 | 4.752 | 1.445 | 21.987 | 10.394 |
| 100 | 5.679 | 1.923 | 46.322 | 12.086 |

For both LCS and Knapsack, tabulation was faster than memoization at the larger tested sizes. At size 100, LCS tabulation took approximately 1.923 ms versus 5.679 ms for memoization; Knapsack tabulation took 12.086 ms versus 46.322 ms for memoization. The memory measurements show the same direction in these runs: at size 100, memoized Knapsack used about 7,072.8 KiB of peak additional traced memory, while tabulation used about 805.8 KiB. The result is workload- and implementation-dependent and does not establish that one approach is always superior.

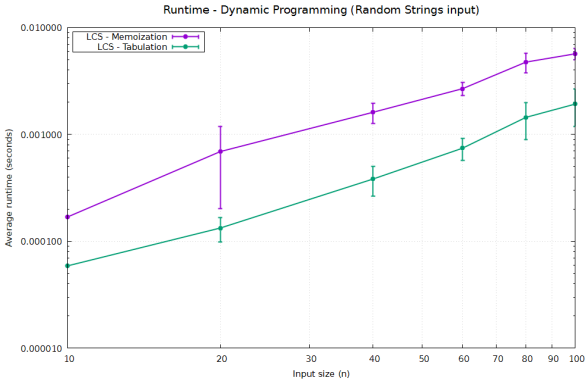



Figure 6.7: LCS runtime scaling.

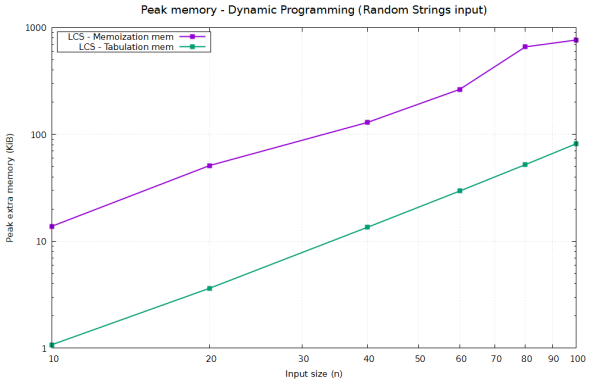



Figure 6.8: LCS peak additional memory.

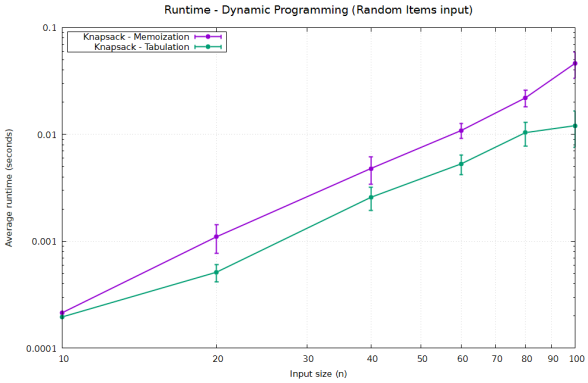



Figure 6.9: Knapsack runtime scaling.

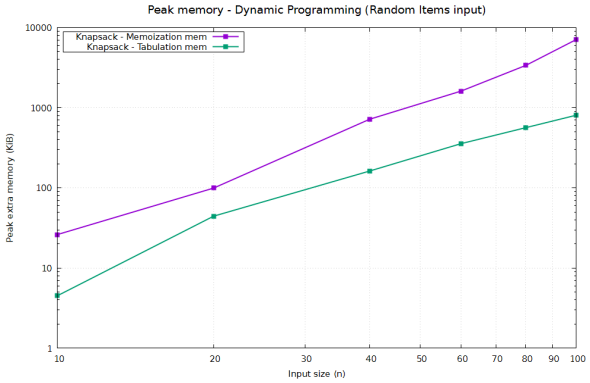



Figure 6.10: Knapsack peak additional memory.

## 6.6 Matrix Multiplication

Standard and Strassen multiplication were run on pairs of random square matrices of dimensions 32, 64, and 128. Table 6.5 reports average runtime in milliseconds and peak additional traced memory in KiB.

Table 6.5: Matrix multiplication measurements (five trials per point).

| Dimension | Standard (ms) | Strassen (ms) | Standard mem. (KiB) | Strassen mem. (KiB) |
|---|---|---|---|---|
| 32 | 4.751 | 4.603 | 38.2 | 64.8 |
| 64 | 30.536 | 31.623 | 165.5 | 656.6 |
| 128 | 249.948 | 272.290 | 650.7 | 2,533.3 |

Strassen was marginally faster at dimension 32, but standard multiplication was faster at 64 and 128 in these measurements. At dimension 128, Strassen was about 9% slower and used approximately 3.9 times the peak additional traced memory. This is a useful practical crossover

observation: the lower asymptotic exponent of Strassen does not guarantee a speedup at the modest matrix sizes tested, especially when recursion and temporary matrix allocations have non-trivial overhead. Larger dimensions would be required to locate a crossover point for this implementation and machine.

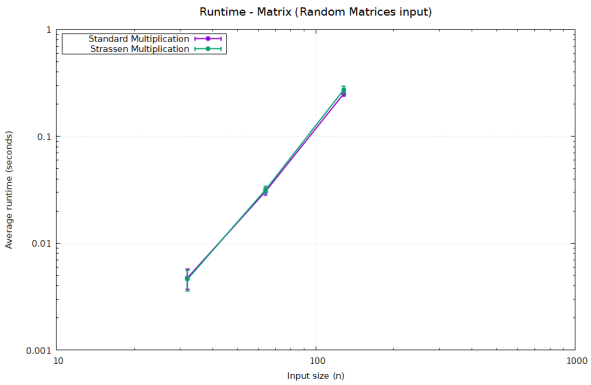



Figure 6.11: Matrix multiplication runtime scaling.

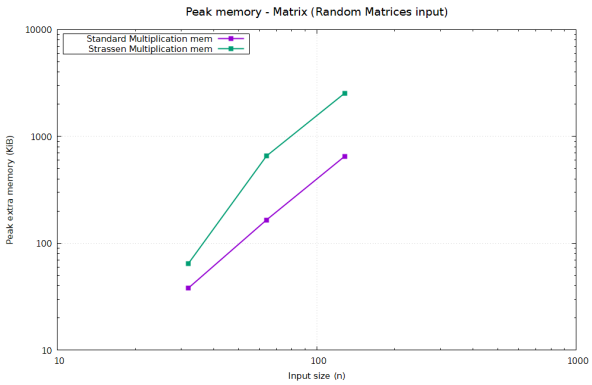



Figure 6.12: Matrix multiplication peak additional memory.

## 6.7 Profiler Observations

The supplied `cProfile`/`pstats` text reports show algorithm functions and, in some captures, activity from the Streamlit host process. Examples of algorithm-level observations include:

- Quick Sort at input size 5,000 recorded 5,971 recursive `sort` calls (with one outer invocation) and approximately 0.020 seconds cumulative time in the profiled algorithm call.
- Knapsack memoization at input size 100 recorded 45,463 recursive `solve` calls (including the outer call) and 44,869 calls to `max`, illustrating the repeated state evaluation and comparison activity in that implementation.
- The Strassen profile at dimension 128 recorded 57 calls to `_strassen_recursive`; it also showed repeated base-case calls to standard multiplication and matrix addition/subtraction helpers.

**Profiler caveat:** some profile files include Streamlit watcher, asyncio, and background-thread events in their cumulative-time ordering. For example, the Insertion Sort profile at size 5,000 includes substantial watcher and event-loop activity alongside the `insertion_sort` function. Therefore, the profile tables are useful for inspecting function calls, but cumulative times from a Streamlit-hosted profile should not be attributed wholly to the selected algorithm. A future improvement would be to run profiling in a separate worker process to isolate application-server overhead.

## 6.8 Overall Findings

Across the supplied scaling experiments, the observed trends broadly align with the expected complexity patterns: Insertion Sort and Linear Search increase more sharply with input size than their more efficient alternatives; LCS and Knapsack runtimes rise with their workload sizes; and graph runtimes increase as vertices are added. The matrix experiment did not reach a size at which Strassen outperformed standard multiplication. The plots also show that time and memory are separate dimensions: an implementation that is competitive in runtime may allocate more temporary memory.

These results are five-trial measurements from one Windows/CPython 3.14 environment. Small timings, especially microsecond-scale search operations, are sensitive to system scheduling and measurement overhead. They should be used as a reproducible project demonstration, not as hardware-independent performance claims.

# Chapter 7

# Limitations and Future Work

## 7.1 Current Limitations

- GNUplot rendering requires a separately installed system executable.
- `tracemalloc` measures traced Python allocations, not all process memory or native allocations.
- The native gprof example requires a supported GCC/gprof environment; Windows users are advised to use WSL.
- Benchmark values are machine- and workload-dependent.
- The dashboard's browser-level interaction should be rechecked on the final presentation machine, since browser dimensions and deployment environments can differ from the development laptop.

## 7.2 Future Work

Potential extensions include additional algorithm families, more dataset distributions, persistent profile browsing by experiment, exportable PDF benchmark reports, native C/C++ algorithm implementations, and automated benchmark comparisons over multiple machines.

# Chapter 8

# Conclusion

AlgoBench combines algorithm implementations, reproducible dataset generation, a reusable benchmark engine, memory measurement, function profiling, CSV history, and visualization in one modular application. Its registry-driven design keeps algorithms separate from benchmarking mechanics and allows the dashboard to invoke different algorithms through a common interface. The integrated tests validate all sixteen registered implementations through the benchmark engine. The final demonstration should record fresh machine-specific measurements and show the full dashboard workflow.

# Team Contributions

| Member | Contribution |
|---|---|
| CS25BT005 | Sorting and searching implementations and their unit tests. |
| CS25BT022 | Graph algorithms, graph dataset generator, and tests. |
| CS25BT031 | LCS, Knapsack, matrix multiplication, string/matrix generators, and tests. |
| CS25BT041 | Generic benchmark engine, runtime/memory measurement, cProfile/g-prof support, GNUplot scripts, and engine tests. |
| CS25BT051 | Streamlit dashboard, integration, result browsing, automation, and final project packaging. |

# References


1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

2. Python Software Foundation, *Python Documentation*, https://docs.python.org/.

3. Streamlit, *Streamlit Documentation*, https://docs.streamlit.io/.

4. GNU Project, *GNUplot Documentation*, https://www.gnuplot.info/documentation.html.

# AI Tool Usage Declaration

The team used ChatGPT for assistance with integration planning, dashboard and benchmark-engine integration, debugging guidance, and drafting the project report and presentation. Claude was used to produce a review and testing report for the integrated application. The team ran the application locally across the five algorithm domains and collected the CSV measurements and GNUplot figures included in this report. The team remains responsible for reviewing the complete submitted code, understanding the algorithms and measurement methodology, and being able to explain all AI-assisted changes during the project evaluation. Each member should verify the description above and amend it if any additional AI assistance needs to be declared under the course instructions.